ARTIFICIAL INTELLIGENCE & MACHINE LEARNINGLABORATORY (Effective from the academic year 2023-2024) SEMESTER – VI			
Subject Code	22CSL67	**CIE Marks**	50 Marks
Number of Lecture Hours/Week	2	**SEE Marks**	50 Marks
Total Number of Lecture Hours	30	**Exam Hours**	3 hours
CREDITS – 01			

Course Learning objectives:

- Understand Fundamentals of State space search representation.
- Study problem reduction techniques.
- Learn hypothesis space search.
- Acquire the knowledge of machine learning algorithms.

Experiments:

1. Implement ao* search algorithm.
2. Implement and demonstrate the travelling salesman problem
3. write a program to iimplement alpha-beta pruning using python
4. write a program to implement tic-tac-toe game using python
5. write a program to implement HILL climbing algorithm.
6. Implement resolution principle on FOPL related problem.
7. Build an Artificial Neural Network by implementing the Back propagation algorithm and test the same using appropriate data sets.
8. Implement 8- Queens algorithm
9. write a program to implement k-nearest neighbour algorithm to classify the iris data set. print both correct and wrong predictions. java/python ml library classes can be used for this problem.
10. Implement the non-parametric locally weighted regression algorithm in order to fit data points. select appropriate data set for your experiment and draw graphs.

	Course outcomes
CO1	Apply Problem-Solving Techniques on state space search.
CO2	Implement Game Playing Algorithms.
CO3	Demonstrate Machine Learning Algorithms for Classification and Hypothesis Learning.
CO4	Apply and Compare Clustering Algorithms for Data Analysis.
CO5	Implement Non-Parametric Regression Techniques for Data Fitting.

1.Implement AO* search algorithm.

```
class Node:

    def __init__(self, name):

        self.name = name

self.heuristic = 0

self.children = []

self.status = 'unvisited'

self.solution = None


    def __str__(self):

        return f"{self.name}"



# AO* Search Algorithm

def ao_star(node):

print(f"Visiting Node: {node}")
```

```
    if not node.children:
        return node.heuristic

min_cost = float('inf')
best_path = None
    for group in node.children:
        cost = sum(child.heuristic for child in group)
        if cost <min_cost:
min_cost = cost
best_path = group

node.heuristic = min_cost
node.solution = best_path

    for child in best_path:
        if child.status == 'unvisited':
ao_star(child)

node.status = 'solved'
    return node.heuristic

def print_solution(node):
    if not node.solution:
        print(f"{node}", end="")
        return
    print(f"{node} -> [", end="")
```

```
    for i, child in enumerate(node.solution):
        if i> 0:
print(" AND ", end="")
print_solution(child)
print("]", end="")

# Example Graph (AND-OR Tree)
# A -> [[B, C], [D]]
# B -> [[E]]
# C -> []
# D -> [[F]]
# E, F -> []

# Creating Nodes
A = Node("A")
B = Node("B")
C = Node("C")
D = Node("D")
E = Node("E")
F = Node("F")

# Assigning heuristic values
E.heuristic = 2
F.heuristic = 3
C.heuristic = 4


# Defining children (AND-OR branches)
A.children = [[B, C], [D]]
```

```
B.children = [[E]]

D.children = [[F]]


# Run AO* and print solution

ao_star(A)

print("\nOptimal Solution Path:")

print_solution(A)
```

OUTPUT:Visiting Node: A

Visiting Node: B

Visiting Node: E

Visiting Node: D

Visiting Node: F

Optimal Solution Path:

A -> [B -> [E] AND C]

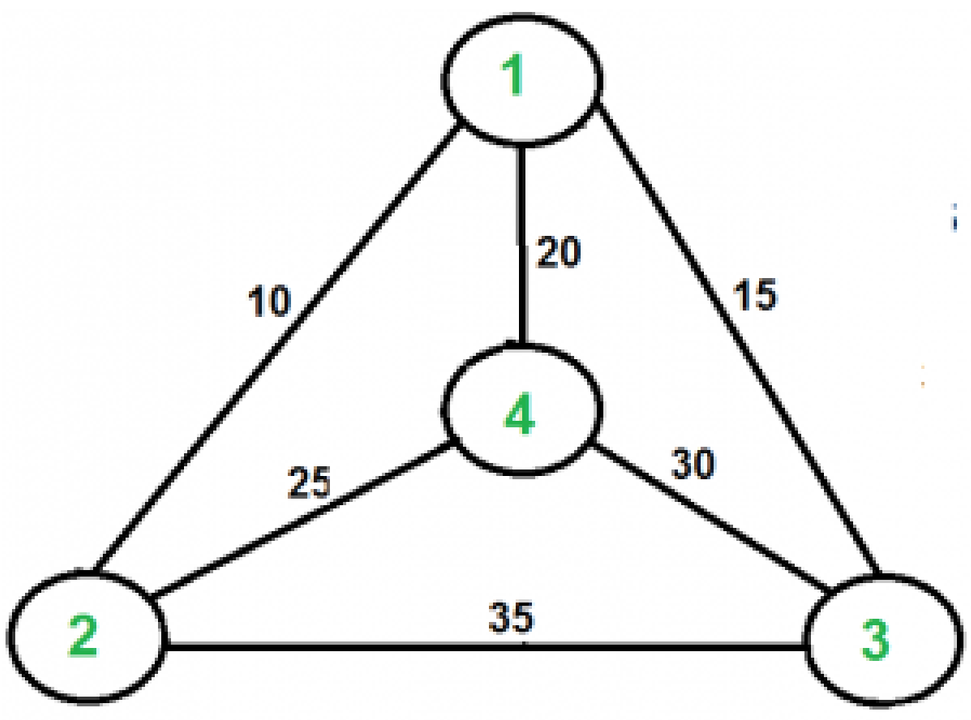
2. Implement and demonstrate Travelling salesman problem.

```
import numpy as np

def travellingsalesman(c):
```

```
    global cost
adj_vertex = 999
min_val = 999
    visited[c] = 1
print((c + 1), end=" ")
for k in range(n):
        if (tsp_g[c][k] != 0) and (visited[k] == 0):
            if tsp_g[c][k] <min_val:
min_val = tsp_g[c][k]
adj_vertex = k
    if min_val != 999:
        cost = cost + min_val
    if adj_vertex == 999:
adj_vertex = 0
print((adj_vertex + 1), end=" ")
        cost = cost + tsp_g[c][adj_vertex]
        return
travellingsalesman(adj_vertex)
n = 4
cost = 0
visited = np.zeros(n, dtype=int)
tsp_g = np.array([[0, 10, 15, 20],
                  [10, 0, 35, 25],
                  [15, 35, 0, 30],
                  [20, 25, 30, 0]])
print("Shortest Path:", end=" ")
travellingsalesman(0)
print()
print("Minimum Cost:", end=" ")
print(cost)
```

OUTPUT:

Given graph:-



Result:-

Shortest Path: 1 2 4 3 1

Minimum Cost: 80

3.Write a program to iimplement Alpha-Beta Pruning using Python

```
import math
def alpha_beta_pruning(depth, node_index, maximizing_player, values, alpha, beta):
    # Base case: leaf node
    if depth == 0 or node_index>= len(values):
        return values[node_index]

    if maximizing_player:
        max_eval = -math.inf
        for i in range(2):  # Each node has two children
            eval = alpha_beta_pruning(depth - 1, node_index * 2 + i, False, values, alpha,
beta)
            max_eval = max(max_eval, eval)
            alpha = max(alpha, eval)
            if beta <= alpha:
                break  # Beta cutoff
        return max_eval
    else:
        min_eval = math.inf
        for i in range(2):  # Each node has two children
            eval = alpha_beta_pruning(depth - 1, node_index * 2 + i, True, values, alpha,
beta)
            min_eval = min(min_eval, eval)
            beta = min(beta, eval)
            if beta <= alpha:
                break  # Alpha cutoff
        return min_eval

# Example usage
if __name__ == "__main__":
    # Leaf node values for a complete binary tree
    values = [3, 5, 6, 9, 1, 2, 0, -1]
    depth = 3  # Height of the tree
```

```
optimal_value = alpha_beta_pruning(depth, 0, True, values, -math.inf, math.inf)
print(f"The optimal value is: {optimal_value}")
```

OUTPUT :

The optimal value is: 5

4. Write a program to implement Tic-Tac-Toe game using python

```
def print_board(board):
    """ Print the current state of the Tic-Tac-Toe board """
    for row in board:
print(" | ".join(row))
print("-" * 9)


def check_winner(board, player):
    """ Check if the specified player has won the game """
    for row in board:
        if all(cell == player for cell in row):
            return True
    for col in range(3):
        if all(board[row][col] == player for row in range(3)):
            return True
if all(board[i][i] == player for i in range(3)):
    return True
if all(board[i][2-i] == player for i in range(3)):
    return True
return False
```

```
def is_full(board):
    """ Check if the board is completely filled """
    return all(cell != ' ' for row in board for cell in row)

def tic_tac_toe():
    """ Main function to run the Tic-Tac-Toe game """
    board = [[' ' for _ in range(3)] for _ in range(3)]
current_player = 'X'

    while True:
print_board(board)
print(f"Player {current_player}'s turn.")
        row = int(input("Enter row (1-3): "))
        col = int(input("Enter column (1-3): "))
        row -= 1
        col -= 1
        if board[row][col] == ' ':
            board[row][col] = current_player
        else:
print("Invalid move! Try again.")
            continue

        # Check if the current player has won
        if check_winner(board, current_player):
print_board(board)
print(f"Player {current_player} wins!")
            break

        # Check if the board is full (tie game)
        if is_full(board):
print_board(board)
print("It's a tie!")
            break
```

```
        # Switch to the other player
current_player = 'O' if current_player == 'X' else 'X'

if __name__ == "__main__":
tic_tac_toe()
```

OUTPUTS

```
  |   |
---------
  |   |
---------
  |   |
---------
```

Player X's turn.

Enter row (1-3): 2

Enter column (1-3): 2

```
  |   |
---------
  | X |
---------
  |   |
---------
```

Player O's turn.

Enter row (1-3): 1

Enter column (1-3): 1

```
O |   |
---------
  | X |
---------
  |   |
---------
```

Player X's turn.

Enter row (1-3): 1

Enter column (1-3): 2

O | X |

  | X |

  | |

Player O's turn.

Enter row (1-3): 3

Enter column (1-3): 2

O | X |

  | X |

  | O |

Player X's turn.

Enter row (1-3): 1

Enter column (1-3): 3

O | X | X

  | X |

  | O |

Player O's turn.

Enter row (1-3): 3

Enter column (1-3): 3

O | X | X

  | X |

  | O | O

Player X's turn.

Enter row (1-3): 3

Enter column (1-3): 1

O | X | X

** | X |**

X | O | O

Player X wins!

5. write a program to implement Hill Climbing algorithm

```
import random

# Distance matrix representing distances between cities
# Replace this with the actual distance matrix for your problem
distance_matrix = [
    [0, 10, 15, 20],
    [10, 0, 35, 25],
    [15, 35, 0, 30],
    [20, 25, 30, 0]
]
def total_distance(path):
    # Calculate the total distance traveled in the given path
    total = 0
    for i in range(len(path) - 1):
        total += distance_matrix[path[i]][path[i+1]]
    total += distance_matrix[path[-1]][path[0]]  # Return to starting city
    return total
def hill_climbing_tsp(num_cities, max_iterations=10000):
    current_path = list(range(num_cities))  # Initial solution, visiting cities in order
    current_distance = total_distance(current_path)
    for _ in range(max_iterations):
        # Generate a neighboring solution by swapping two random cities
        neighbor_path = current_path.copy()
i, j = random.sample(range(num_cities), 2)
neighbor_path[i], neighbor_path[j] = neighbor_path[j], neighbor_path[i]
neighbor_distance = total_distance(neighbor_path)

# If the neighbor solution is better, move to it
if neighbor_distance < current_distance:
    current_path = neighbor_path
    current_distance = neighbor_distance
```

```
    return current_path
def main():
    num_cities = 4  # Number of cities in the TSP
    solution = hill_climbing_tsp(num_cities)
    print("Optimal path:", solution)
    print("Total distance:", total_distance(solution))
if __name__ == "__main__":
    main()
```

OUTPUT

Optimal path: [1, 0, 2, 3]

Total distance: 80

6. Implement Resolution principle on FOPL related Problem.

```
def negate_literal(literal):
    """ Negate a literal by adding or removing the negation symbol '~' """
    if literal.startswith('~'):
        return literal[1:]  # Remove negation
    else:
        return '~' + literal  # Add negation

def resolve(clause1, clause2):
    """ Resolve two clauses to derive a new clause """
    new_clause = []
    resolved = False

    # Copy literals from both clauses
    for literal in clause1:
        if negate_literal(literal) in clause2:
            resolved = True
        else:
            new_clause.append(literal)

    for literal in clause2:
        if negate_literal(literal) not in clause1:
```

```
            new_clause.append(literal)

    if resolved:
        return new_clause
    else:
        return None  # No resolution possible

def resolution(propositional_kb, query):
    """ Use resolution to prove or disprove a query using propositional logic """
    kb = propositional_kb[:]
    kb.append(negate_literal(query))  # Add negated query to knowledge base

    while True:
        new_clauses = []
        n = len(kb)
        resolved_pairs = set()  # Track resolved pairs to avoid redundant resolutions

        for i in range(n):
            for j in range(i + 1, n):
                clause1 = kb[i]
                clause2 = kb[j]

                if (clause1, clause2) not in resolved_pairs:
                    resolved_pairs.add((clause1, clause2))
                    resolvent = resolve(clause1, clause2)

                    if resolvent is None:
                        continue  # No resolution possible for these clauses

                    if len(resolvent) == 0:
                        return True  # Empty clause (contradiction), query is proved

                    if resolvent not in new_clauses:
                        new_clauses.append(resolvent)

        if all(clause in kb for clause in new_clauses):
            return False  # No new clauses added, query cannot be proven

        kb.extend(new_clauses)  # Add new clauses to the knowledge base
```

```
# Example usage:
if __name__ == "__main__":
    # Example propositional knowledge base (list of clauses)
    propositional_kb = [
        ['~P', 'Q'],
        ['P', '~Q', 'R'],
        ['~R', 'S']
    ]

    # Example query to prove/disprove using resolution
    query = 'S'

    # Use resolution to prove or disprove the query
    result = resolution(propositional_kb, query)

    if result:
        print(f"The query '{query}' is PROVED.")
    else:
        print(f"The query '{query}' is DISPROVED.")
```

OUTPUT

The query 'S' is DISPROVED.

7. Build an Artificial Neural Network by implementing the Back propagation algorithm and test the same using appropriate data sets.

- ***BACKPROPAGATION ALGORITHM:***

This algorithm operates in two phases: forward propagation and backward propagation:

1) **Forward propagation:**

1. **Takes inputs as a matrix (2D array of numbers)**
2. **Multiplies the input by a set weights (performs a dot product aka matrix multiplication)**
3. **Applies an activation function called Sigmoid function.**

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

4. **Returns an output.**

2) **Backward propagation:**

1. **Error is calculated by taking the difference from the desired output from the data and the predicted output.**
2. **The weights are then altered slightly according to the error using Derivative sigmoid function.**

$$\sigma'(x) = \frac{d}{dx}\sigma(x) = \sigma(x)(1 - \sigma(x))$$

3) **To train the neural network, this process is repeated 1,000+ times. The more the data is trained upon, the more accurate our outputs will be.**

- ***INPUT DATA SET:***

Training Examples:

Example	Sleep	Study	Expected % in Exams
1	2	9	92
2	1	5	86
3	3	6	89

Normalize the input

Example	Sleep	Study	Expected % in Exams
1	2/3 = 0.66666667	9/9 = 1	0.92
2	1/3 = 0.33333333	5/9 = 0.55555556	0.86
3	3/3 = 1	6/9 = 0.66666667	0.89

- ***PROGRAM:***

```
import numpy as np

# X = (hours sleeping, hours studying), y = score on test X = np.array(([2, 9], [1, 5], [3, 6]), dtype=float)

y = np.array(([92], [86], [89]), dtype=float)



# scale units

X = X/np.amax(X, axis=0) # maximum of X array y = y/100 # max test score is 100

class Neural_Network(object):

def _init_(self):

#parameters self.inputSize = 2

self.outputSize = 1

self.hiddenSize = 3



#weights

self.W1 = np.random.randn(self.inputSize, self.hiddenSize) # (3x2) weight matrix from input to hidden layer self.W2 = np.random.randn(self.hiddenSize, self.outputSize) # (3x1) weight matrix from hidden to output layer



def forward(self, X):

#forward propagation through our network

self.z = np.dot(X, self.W1) # dot product of X (input) and first set of 3x2 weights self.z2 = self. sigmoid(self.z) # activation function

self.z3 = np.dot(self.z2, self.W2) # dot product of hidden layer (z2) and second set of 3x1 weights o = self.sigmoid(self.z3) # final activation function

return o
```

```
def sigmoid(self, s):

# activation function return 1/(1+np.exp(-s))


def sigmoidPrime(self, s):

#derivative of sigmoid return s * (1 - s)


def backward(self, X, y, o):
# backward propgate through the network self.o_error = y - o # error in output

self.o_delta = self.o_error*self.sigmoidPrime(o) # applying derivative of sigmoid to error


self.z2_error = self.o_delta.dot(self.W2.T) # z2 error: how much our hidden layer weights contributed to output error

self.z2_delta = self.z2_error*self.sigmoidPrime(self.z2) # applying derivative of sigmoid to z2 error


self.W1 += X.T.dot(self.z2_delta) # adjusting first set (input --> hidden) weights self.W2 += self.z2.T.dot(self.o_delta) # adjusting second set (hidden --> output) weights
```

```
def train (self, X, y):

o = self.forward(X) self.backward(X, y, o)


NN = Neural_Network()

for i in range(1000): # trains the NN 1,000 times NN.train(X, y)
```

```
print ("Input: \n" + str(X))

print (" Actual Output: \n" + str(y))

print (" Predicted Output: \n" + str(NN.forward(X)))
```

- ***OUTPUT:***

```
Input:
[[0.66666667 1.        ]
 [0.33333333 0.55555556]
 [1.         0.66666667]]

Actual Output:
[[0.92]
 [0.86]
 [0.89]]
```

Predicted output: [0.90649194]

[0.85676141]

[0.90800106]

---

8. Write a Program to implement 8-Queens Problem using Python.

```
global N
N = 8

def printSolution(board):
    for i in range(N):
        for j in range(N):
            print(board[i][j], end=' ')
        print()

def isSafe(board, row, col):
    # Check this row on left side
    for i in range(col):
        if board[row][i] == 1:
            return False

    # Check upper diagonal on left side
    for i, j in zip(range(row, -1, -1), range(col, -1, -1)):
        if board[i][j] == 1:
            return False

    # Check lower diagonal on left side
```

```
    for i, j in zip(range(row, N, 1), range(col, -1, -1)):
        if board[i][j] == 1:
            return False

    return True

def solveNQUtil(board, col):
    # base case: If all queens are placed
    # then return true
    if col >= N:
        return True

    for i in range(N):
        if isSafe(board, i, col):
            # Place this queen in board[i][col]
            board[i][col] = 1

            # recur to place rest of the queens
            if solveNQUtil(board, col + 1) == True:
                return True

            board[i][col] = 0

    return False

def solveNQ():
    board = [[0]*N for _ in range(N)]

    if solveNQUtil(board, 0) == False:
        print("Solution does not exist")
        return False
```

```
    printSolution(board)
    return True

# driver program to test above function
solveNQ()
```

OUTPUT

0 0 0 0 1 0 0 0

0 0 0 0 0 0 0 1

0 0 0 1 0 0 0 0

0 0 0 0 0 1 0 0

0 1 0 0 0 0 0 0

0 0 0 0 0 0 1 0

0 0 1 0 0 0 0 0

1 0 0 0 0 0 0 0

9. write a program to implement k-nearest neighbour algorithm to classify the iris data set. Print both correct and wrong predictions. java/python ml library classes can be used for this problem.

 # PROGRAM:

```
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import confusion_matrix
from sklearn.metrics import accuracy_score

# Load the iris dataset
```

```
iris = datasets.load_iris()
X=iris.data
y=iris.target

# Split the iris dataset into 75%-train data(112 samples) and 25%-test data(38
samples)
X_train,X_test,y_train,y_test=train_test_split(X,y,random_state=0,test_size=0.25)
# Print the total number of samples in the dataset and random samples taken as test
data
print("Size of full iris-dataset: 150")
print("Size of train dataset: 112")
print("Size of test dataset: 38")

#Build k-Nearest Neighbour model
classifier=KNeighborsClassifier(n_neighbors=5,p=2,metric='euclidean')
classifier.fit(X_train,y_train)

#predict the test resuts and print confusion matrix
y_pred=classifier.predict(X_test)
cm=confusion_matrix(y_test,y_pred)
print('\nConfusion matrix is as follows:\n',cm)

#print Accuracy Metrics with correct and wrong predictions
print('\nAccuracy of the classifer is: ',accuracy_score(y_test,y_pred))
print(" \ncorrect predicitions: ",accuracy_score(y_test,y_pred))
print("  worng predicitions: ",(1-accuracy_score(y_test,y_pred)))
```

 # OUTPUT:

```
Size of full iris-dataset: 150
Size of train dataset: 112
```

Size of test dataset: 38

Confusion matrix is as follows:

[[13 0 0]

[0 15 1]

[0 0 9]]

Accuracy of the classifer is: 0.9736842105263158

correct predicitions: 0.9736842105263158

worng predicitions: 0.02631578947368418

---

10. Implement the non-parametric Locally Weighted Regression algorithm in order to fit data points. Select appropriate data set for your experiment and draw graphs.

```
# PROGRAM: import matplotlib.pyplot as plt
import pandas as pd
import numpy as np

# Kernel function for calculating the weights
def kernel(point, xmat, k):
    m, n = np.shape(xmat)
    weights = np.mat(np.eye(m))  # Create a diagonal matrix with ones
    for j in range(m):
        diff = point - xmat[j]  # Difference between the point and each row of xmat
        weights[j, j] = np.exp(diff * diff.T / (-2.0 * k**2))  # Compute the Gaussian kernel
    return weights

# Local weight calculation for a single point
def localWeight(point, xmat, ymat, k):
    wei = kernel(point, xmat, k)
    # Solve for W using the normal equation: W = (X.T * W * X)^-1 * X.T * W * Y
    W = np.linalg.inv(xmat.T @ (wei @ xmat)) @ (xmat.T @ (wei @ ymat))
    return W

# Locally weighted regression for all data points
def localWeightRegression(xmat, ymat, k):
    m, n = np.shape(xmat)
    ypred = np.zeros(m)
    for i in range(m):
        ypred[i] = xmat[i] @ localWeight(xmat[i], xmat, ymat, k)  # Predict for each point
```

```
    return ypred


# Load data points
data = pd.read_csv('Tips.csv')


# Print column names to verify
print("Columns in the dataset:", data.columns)


# Ensure that the column names are correct or match the CSV
bill = np.array(data['total_bill'])  # Use the correct column name
tip = np.array(data['tip'])  # Use the correct column name


# Preparing the matrix X with a column of ones for the intercept term
mbill = np.mat(bill).T
mtip = np.mat(tip).T
m = np.shape(mbill)[0]
one = np.mat(np.ones(m)).T  # Create a column of ones for intercept
X = np.hstack((one, mbill))  # Add the column of ones as the first column in X


# Perform locally weighted regression for k=3 and k=9
k3_predictions = localWeightRegression(X, mtip, k=3)
k9_predictions = localWeightRegression(X, mtip, k=9)


# Convert X to a numpy array for proper indexing
X_array = np.array(X)


# Sort the data by bill to plot a smooth line
SortIndex = X_array[:, 1].argsort()  # Sort by the 'bill' column
```

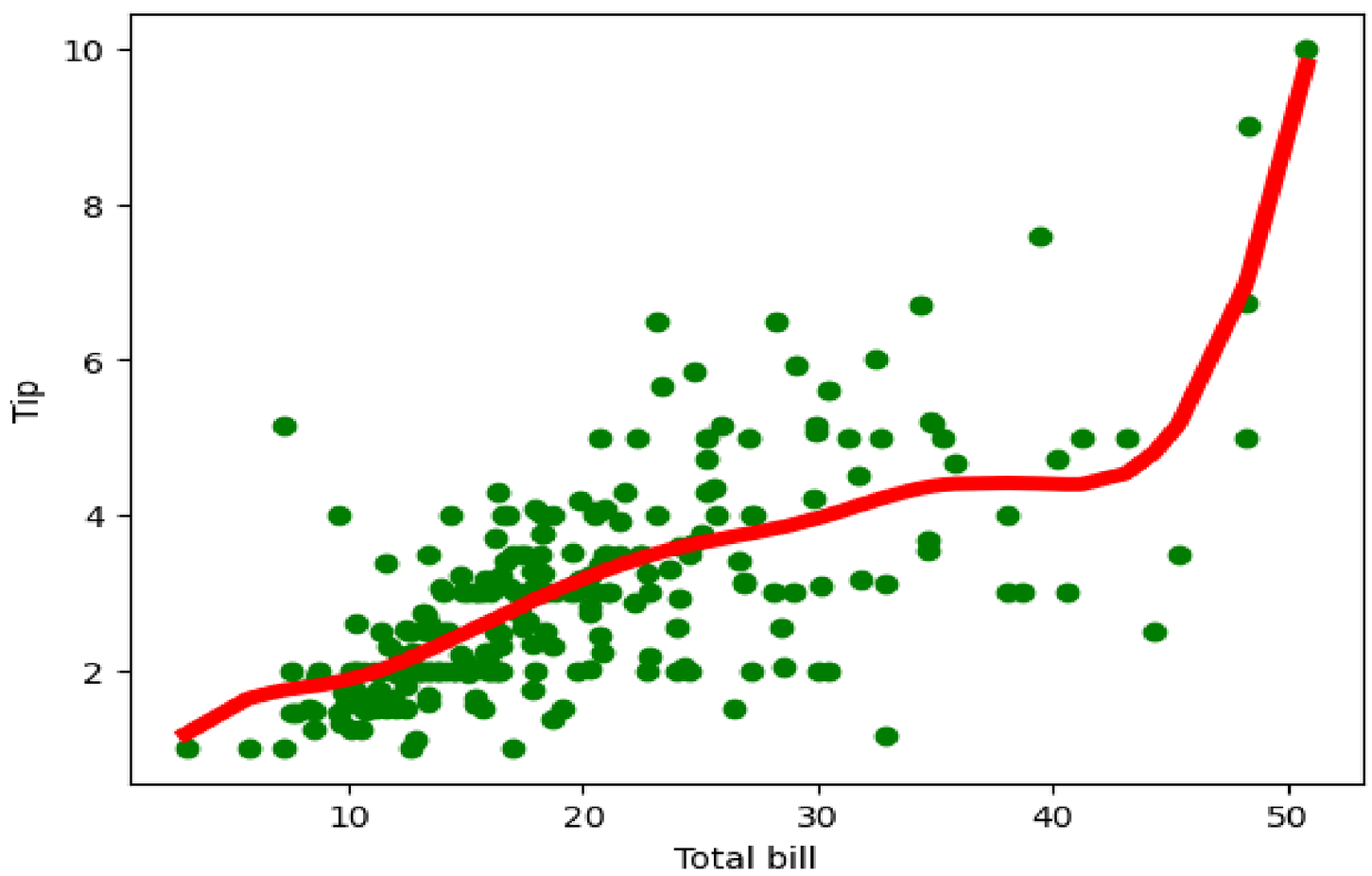
```
xsort = X_array[SortIndex][:, 1]  # Sorted values of the bill
k3_sorted = k3_predictions[SortIndex]  # Sorted predictions for k=3
k9_sorted = k9_predictions[SortIndex]  # Sorted predictions for k=9


# Create figure for k=3
fig1 = plt.figure()
ax1 = fig1.add_subplot(1, 1, 1)
ax1.scatter(bill, tip, color='green', label='Original Data')  # Scatter plot of the original
data
ax1.plot(xsort, k3_sorted, color='red', linewidth=2, label='k=3')  # Regression line for
k=3
ax1.set_xlabel('Total Bill')
ax1.set_ylabel('Tip')
ax1.set_title('Locally Weighted Regression for k=3')
ax1.legend()


# Create figure for k=9
fig2 = plt.figure()
ax2 = fig2.add_subplot(1, 1, 1)
ax2.scatter(bill, tip, color='green', label='Original Data')  # Scatter plot of the original
data
ax2.plot(xsort, k9_sorted, color='blue', linewidth=2, label='k=9')  # Regression line for
k=9
ax2.set_xlabel('Total Bill')
ax2.set_ylabel('Tip')
ax2.set_title('Locally Weighted Regression for k=9')
ax2.legend()
```

 # Show both figures

plt.show()

 # OUTPUT:

1) **Regression with parameter k=3**



2) **Regression with parameter k=9**